The design of Ruby's object extension is elegant, but you won't find it in Elixir. This isn't necessarily a bad thing though. Functions in Elixir are explicitly invoked with the module name—more on this later—so you know exactly where a function is defined, and you can quickly find its documentation or source code. Every design decision has trade-offs, and Elixir's function-oriented approach trades elegance for explicitness.

Let's quickly go over Elixir's data types. These aren't all of them, but they are the most common ones you'll use. Some are simple, but others require a bit more explanation:

Atom
: Atoms are named constants expressed with a colon (`:an_atom`). They are like symbols in Ruby.

Boolean
: Booleans are special atoms that represent truth (`true` or `false`).

Nil
: `nil` is a special atom that represents the absence of a value.

Integer
: Integers are expressed with digits (`123`) or via alternative base representations (`0xA`). You can add `_` to integers to visually break up large numbers (`1_000_000`). Integers don't have a maximum size.

Float
: Floats are expressed using digits and a period (`10.53`). They can also be expressed in scientific notation (`1.0e10`). Floats have a maximum size of `1.7976931348623157e+308`.

Bitstring, Binary
: Bitstrings represent a collection of bits. They are expressed using angle brackets (`<<1, 2>>`). Binaries are a type of bitstring with a number of bits divisible by 8. The most common type of binary is a string. Bitstrings can be slightly jarring to see in real-world code—because they're not common in other languages—but they allow for easy interaction with binary data.

String
: Strings are expressed using double quotations, like `"Hello!"`. Strings are easy to work with, as in Ruby, but are actually stored as binary bitstrings under the hood. In practice you won't have to worry about this, but sometimes you'll see function calls like `is_binary(var)`, which is commonly used to check for a string argument.